Карта проекта EH/Unwind Analyzer

Подробный пошаговый разбор: от запуска программы до .pdata, .xdata, UNWIND_INFO, 27 эвристик, отчетов и плана переписывания на C

Этот документ написан как учебник по текущему проекту. Он не предполагает, что ты уже знаешь C, PE или unwind. Сначала идет простая картина, затем проект разбирается слоями: что входит в модуль, что из него выходит, какие структуры проходят дальше и почему именно так.

0. Самая короткая мысль проекта

```
PE-файл
  -> читаем заголовки и секции
  -> находим Exception Directory
  -> читаем .pdata как массив RUNTIME_FUNCTION
  -> каждая RUNTIME_FUNCTION дает BeginAddress, EndAddress, UnwindInfoRVA
  -> по UnwindInfoRVA находим UNWIND_INFO, обычно в .xdata
  -> декодируем UNWIND_INFO
  -> считаем локальные, глобальные и комбинированные эвристики
  -> пишем console, CSV, Ghidra CSV, JSON, HTML
```

Если совсем коротко: `.pdata` показывает, где функции и где искать их unwind-описания. `.xdata` обычно хранит сами `UNWIND_INFO`. Эвристики проверяют, похожа ли эта связка на нормальный результат компилятора или на повреждение, обфускацию, перехват исключений, ручную правку метаданных или защитный механизм.

1. Представь exe как книгу

Windows PE-файл можно представить как книгу. В книге есть главы, оглавление и служебные инструкции. Машинный код - это главы. `.pdata` - это оглавление функций. `.xdata` - это инструкции для Windows, как восстановить стек, если исключение произошло внутри функции.

Часть	Простая аналогия	Что там лежит
`.text`	Главы книги	Машинный код функций
`.pdata`	Оглавление функций	Записи `RUNTIME_FUNCTION` по 12 байт
`.xdata`	Инструкции к оглавлению	`UNWIND_INFO` для функций
`.rdata`	Справочные данные	Константы и read-only данные

1.1. Что важно не перепутать

- `.pdata` обычно не хранит сами unwind-операции. Она хранит ссылки на них.
- `.xdata` обычно хранит `UNWIND_INFO`, но проект не верит имени секции на слово.
- `UnwindInfoRVA` может указывать не в `.xdata`. Это как раз важный повод для проверки.
- `RVA` - это адрес внутри загруженного образа, а не позиция байта в файле.
- Чтобы прочитать байты по `RVA`, проект должен найти секцию и перевести `RVA` в `file offset`.

2. Термины, без которых код будет непонятен

2.1. PE

`PE` - Portable Executable, формат исполняемых файлов Windows. В этом проекте поддерживается `PE32+ AMD64`, то есть 64-битные Windows-файлы. `PE` состоит из DOS-заголовка, `PE`-заголовка, optional header, таблицы секций и данных секций.

2.2. Секция

Секция - диапазон данных в образе. У секции есть имя, virtual address, virtual size, raw offset, raw size и flags. Важно различать raw-положение в файле и virtual-положение в памяти.

2.3. RVA

RVA - Relative Virtual Address. Это адрес относительно начала образа в памяти. Например, если ImageBase равен 0x140000000, а RVA равен 0x1000, то адрес в памяти обычно 0x140001000. Но в файле на диске этот RVA может лежать совсем в другом месте.

```
RVA -> file offset делается через секцию:

1. Найти секцию, где:
   section.VirtualAddress <= RVA < section.VirtualAddress + section_size
2. Посчитать delta = RVA - section.VirtualAddress
3. file_offset = section.RawPointer + delta
4. Проверить, что delta не выходит за raw_size
5. Проверить, что file_offset внутри файла
```

2.4. Exception Directory

В PE optional header есть массив DataDirectory. Для x64 exception/unwind метаданных нужен элемент IMAGE_DIRECTORY_ENTRY_EXCEPTION. Он дает RVA и размер таблицы. Обычно эта таблица физически лежит в секции .pdata.

2.5. RUNTIME_FUNCTION

RUNTIME_FUNCTION - запись из .pdata. Она описывает один диапазон функции и указывает, где лежит UNWIND_INFO. Каждая запись занимает 12 байт.

Поле	Размер	Смысл
BeginAddress	4 байта	RVA начала функции
EndAddress	4 байта	RVA конца функции
UnwindInfoAddress	4 байта	RVA структуры UNWIND_INFO

2.6. UNWIND_INFO

UNWIND_INFO - компактная инструкция для Windows. Она описывает, что функция сделала со стеком и регистрами в прологе, есть ли handler, есть ли chain, и как восстановить состояние при исключении.

```
Пример идеи:

Функция в начале сделала:
  push rbp
  sub rsp, 40

UNWIND_INFO говорит Windows:
  был сохранен регистр RBP
  стек уменьшен на 40 байт
  при исключении надо вернуть стек и регистры назад
```

3. Полный путь данных: от запуска до отчета

Ниже - общий сценарий одного запуска. Это главный маршрут, по которому данные проходят через проект. Потом каждый шаг будет разобран глубже.

Шаг	Код	Что происходит	Что появляется на выходе
1	main.c	Разбор командной строки	Args
2	util.c	Файл читается целиком в память	Buf + owned memory
3	pe.c	Проверяются PE-заголовки и секции	PeImage
4	unwind.c	Exception Directory читается как .pdata	RuntimeFunction[]
5	unwind.c	Для каждой записи читается UNWIND_INFO	UnwindSummary[] + us_ok[]
6	heuristics.c	Считаются признаки, score, confidence	HeuristicReport
7	dynamic_win.c	Опционально запускается target под отладчиком	DynamicReport
8	report.c	Пишутся console, CSV, JSON, HTML	Файлы отчетов
9	main.c	Освобождается память и возвращается exit code	0, 1 или 2

3.1. Почему проект устроен именно конвейером

Конвейер удобен потому, что каждый слой знает только свою работу. `pe.c` не знает про JSON. `report.c` не парсит байты. `heuristics.c` не открывает файл. `main.c` держит все куски вместе и отвечает за освобождение памяти.

```
Buf
  -> PeImage
      -> RuntimeFunction[]
          -> UnwindSummary[]
              -> HeuristicReport
              -> DynamicReport
                  -> JSON / HTML / CSV
```

4. Карта файлов и строк

Файл	Строки	Главная ответственность
`src/report.c`	1363	Сериализация: CSV, JSON, HTML, итоговая assessment-логика
`src/dynamic_win.c`	1253	Windows Debug API, breakpoints, runtime evidence
`src/heuristics.c`	1044	27 эвристик, score, confidence, verdict
`src/unwind.c`	681	`Чтение .pdata и декодирование UNWIND_INFO`
`src/main.c`	527	CLI, порядок вызовов, владение памятью
`src/util.c`	348	Буфер файла, безопасное чтение, строки, SHA-256
`src/pe.c`	301	`PE-заголовки, секции, RVA mapping`
`src/app_info.c`	68	Версия, компилятор, платформа, timestamp

Если переписывать проект с нуля, не надо начинать с самого большого файла. Самый правильный путь: `util.c` -> `pe.c` -> чтение `.pdata` в `unwind.c` -> минимальный `main.c` -> минимальный report -> эвристики -> динамика.

5. `main.c`: главный сценарий программы

`main.c` - не самый низкий уровень. Он похож на диспетчера: принимает аргументы, вызывает модули по порядку, хранит указатели на выделенную память и в конце все освобождает.

5.1. Структура Args

Args - это собранные настройки запуска. Вместо того чтобы передавать по коду десятки отдельных переменных, `main.c` складывает их в одну структуру.

Поле Args	Что означает
input	Путь к анализируемому PE-файлу
dump	`Нужно ли печатать RUNTIME_FUNCTION entries`
`show_ops`	Печатать ли decoded unwind operations
heuristics	Запускать ли эвристический анализ
`csv_path / ghidra_csv_path`	Куда писать CSV
`json_path / html_path`	Куда писать JSON и HTML
`follow_chain`	`Идти ли по chained UNWIND_INFO`
`strict_abi`	Включить строгие ABI diagnostics
dynamic	Запускать ли Windows dynamic validation
`timeout_ms / trace_steps`	Настройки динамического режима
`child_argv / child_argc`	`Аргументы target после --`

5.2. parse_args пошагово

1. Если аргументов мало, возвращается PARSE_ERR.
2. Если первый аргумент --help, возвращается PARSE_HELP.
3. Если первый аргумент --version, возвращается PARSE_VERSION.

4 Первый обычный аргумент становится input.
5 Дальше цикл идет по оставшимся аргументам.
6 Флаги вроде --dump просто включают bool-поле.
7 Опции с числом вроде --limit читаются через parse_long_range.
8 Опции с путем вроде --json берут следующий аргумент как путь.
9 Если встречается --, все после него считается аргументами target-программы.
10 Неизвестная опция дает ошибку командной строки.

5.3. main пошагово

1 Создается Args a и вызывается parse_args.
2 Если help/version - печатается справка или версия и программа выходит.
3 buf_init_from_file читает весь input в память.
4 Создается `StrBuf` err для сообщений об ошибках.
5 pe_parse заполняет `PeImage` img.
6 unwind_read_runtime_functions_ex читает `.pdata` в RuntimeFunction* rf.
7 Выделяются массивы `UnwindSummary`* us и bool* us_ok.
8 Для каждой RuntimeFunction вызывается unwind_parse.
9 Если --dump, каждая запись печатается через print_one.
10 Если --heuristics, вызывается heuristics_analyze.
11 Если --dynamic, вызывается dynamic_collect_windows.
12 Если запрошены отчеты, готовится ReportMetadata.
13 report_write_csv, report_write_json_ex, report_write_html пишут файлы.
14 В конце main освобождает hr, dr, us_ok, us, rf, img.sections, `StrBuf` и `Buf` memory.

```
Главная идея main.c:

прочитать файл
  -> распарсить PE
  -> прочитать .pdata
  -> распарсить UNWIND_INFO
  -> опционально анализировать
  -> опционально запускать динамику
  -> опционально писать отчеты
  -> все освободить
```

5.4. Почему `main.c` так много освобождает

В C нет автоматического сборщика мусора. Если pe_parse выделил sections, их надо освободить. Если unwind_read_runtime_functions_ex выделил массив rf, его надо освободить. Если sb_init выделил строку, нужен sb_free. Поэтому `main.c` выглядит осторожным: почти у каждого error path есть освобождение уже созданных ресурсов.

6. `util.c`: фундамент проекта

`util.c` - нижний слой. Его задача - сделать опасные операции C чуть безопаснее: чтение байтов с проверкой границ, динамические строки, атомарная запись файлов и SHA-256.

6.1. `Buf`

```
typedef struct Buf {
    const uint8_t* data;
    size_t size;
} Buf;
```

`Buf` не владеет памятью сам по себе. Он только говорит: вот указатель на байты и вот сколько байтов можно читать. Память хранится отдельно в owned_mem.

6.2. buf_init_from_file

1 Открывает файл в binary mode.
2 Через fseek/ftell узнает размер.
3 Выделяет malloc размером file size или 1 байт для пустого файла.

4 Читает файл через fread.
5 Заполняет out->data и out->size.
6 Возвращает указатель на выделенную память через owned_mem.

6.3. buf_read и защита от выхода за границы

Парсер никогда не должен читать байты просто так. Файл может быть битым или специально обрезанным. Поэтому buf_read проверяет: off не больше size, len не больше оставшегося размера, и только потом делает memcpy.

Безопасная идея:

```
if (off > b->size) return false;
if (len > b->size - off) return false;
memcpy(out, b->data + off, len);
```

6.4. Little-endian helpers

PE хранит числа little-endian. Поэтому проект не делает прямой cast байтов к структурам. Он читает 2, 4 или 8 байт и сам собирает число через le16, le32, le64.

6.5. StrBuf

StrBuf - самодельная динамическая строка. В C обычная строка - это char*, но у нее нет встроенного знания о длине и емкости. StrBuf хранит s, len, cap и failed.

Поле	Зачем нужно
s	Указатель на выделенную строку
len	Сколько символов уже записано
cap	Сколько памяти выделено
failed	Флаг, что realloc или append сломался

6.6. file_write_atomic

Отчеты пишутся не сразу в конечный файл. Сначала создается временный файл рядом с целевым, потом он атомарно заменяет старый. Это защищает от ситуации, когда программа упала посередине записи и испортила старый отчет.

7. pe.c: как проект понимает PE-файл

pe.c берет Buf с байтами и превращает его в PeImage. Это значит: проверить заголовки, вытащить ключевые поля, прочитать секции и посчитать структурные проблемы.

7.1. Минимальный порядок PE-парсинга

1 Прочитать первые 2 байта и проверить magic MZ.
2 По offset 0x3c прочитать e_lfanew - смещение PE-заголовка.
3 По e_lfanew проверить сигнатуру PE\0\0.
4 Прочитать COFF header: machine, number of sections, optional header size.
5 Проверить machine == AMD64.
6 Прочитать OptionalHeader magic и проверить PE32+.
7 Прочитать ImageBase, SectionAlignment, FileAlignment, SizeOfImage, SizeOfHeaders.
8 Прочитать NumberOfRvaAndSizes.
9 Прочитать Exception Directory RVA и size.
10 Прочитать таблицу секций.
11 Проверить секции на выходы за границы, пересечения и alignment.
12 Заполнить PeImage.

7.2. Почему проект поддерживает только AMD64

Потому что формат unwind на Windows x64 связан с RUNTIME_FUNCTION и UNWIND_INFO. Для x86 механика исключений другая. ARM64 тоже имеет другую модель. Поэтому проект честно отсекает неподдерживаемые архитектуры.

7.3. PeSection

```
typedef struct PeSection {
    char     name[9];
    uint32_t virtual_address;
    uint32_t virtual_size;
    uint32_t raw_ptr;
    uint32_t raw_size;
    uint32_t characteristics;
} PeSection;
```

name[9] нужен потому, что имя секции в PE занимает 8 байт, а для C-строки нужен еще завершающий ноль.

7.4. Что проверяется по секциям

Проверка	Что означает простыми словами	Зачем это нужно
raw_oob	Секция на диске выходит за размер файла	Файл поврежден или специально странный
virtual_oob	Секция в памяти выходит за SizeOfImage	Образ неконсистентный
raw_overlap	Две секции перекрываются в файле	Может ломать однозначность чтения
virtual_overlap	Две секции перекрываются в памяти	RVA может указывать неоднозначно
misaligned	Нарушены правила выравнивания	Лоадер или инструменты могут вести себя нестандартно

7.5. pe_rva_to_fileoff

Это одна из самых важных функций проекта. Все дальнейшее чтение .pdata и .xdata зависит от нее. Она отвечает на вопрос: если у меня есть RVA, где эти байты лежат в файле?

```
RVA внутри секции:

delta = rva - section.virtual_address
если delta >= section.raw_size -> это virtual tail, в файле байтов нет
file_offset = section.raw_ptr + delta
если file_offset >= file_size -> ошибка

RVA внутри headers:
если rva < SizeOfHeaders -> file_offset = rva
```

8. .pdata: как проект читает таблицу функций

После pe_parse проект знает Exception Directory: img.exception_rva и img.exception_size. Для Windows x64 это таблица RUNTIME_FUNCTION. Обычно она лежит в .pdata, но код ориентируется на RVA из PE-заголовка, а не только на имя секции.

8.1. unwind_read_runtime_functions_ex

1. Если exception_rva == 0 и exception_size == 0, таблицы нет - это не обязательно ошибка.
2. Если одно из этих полей ноль, а другое нет, это malformed directory.
3. exception_rva переводится в file offset через pe_rva_to_fileoff.
4. Проверяется, что exception directory целиком лежит внутри файла.
5. Проверяется, что размер хотя бы 12 байт.
6. Проверяется, что размер кратен 12.
7. n = exception_size / 12.
8. Выделяется массив RuntimeFunction[n].
9. Для каждой записи читаются begin_rva, end_rva, unwind_rva.
10. Массив возвращается наружу, caller обязан сделать free.

8.2. Почему запись 12 байт

Смещение в записи	Поле	Размер	Что значит
+0	`BeginAddress`	4	`RVA начала функции`
+4	`EndAddress`	4	`RVA конца функции`
+8	UnwindInfoAddress	4	`RVA UNWIND_INFO`

8.3. Пример одной записи

```
begin_rva  = 0x00001000
end_rva    = 0x00001120
unwind_rva = 0x00003040

Читается так:
  функция занимает диапазон [0x1000, 0x1120)
  ее UNWIND_INFO лежит по RVA 0x3040
```

Запись использует half-open range: начало входит, конец не входит. Это обычный способ описывать диапазоны в системном коде.

9. `.xdata` и `UNWIND_INFO`: что проект читает дальше

Для каждой `RUNTIME_FUNCTION` проект берет unwind_rva и вызывает unwind_parse. `UNWIND_INFO` обычно находится в `.xdata`, но проект проверяет реальное положение через pe_find_section_for_rva. Если метаданные лежат в `.text`, writable-секции или неизвестной секции, это может стать finding.

9.1. Заголовок `UNWIND_INFO`

Первые 4 байта `UNWIND_INFO` содержат packed-поля. Код не читает C-структуру, а вручную извлекает биты.

Байт/биты	Поле	Смысл
byte0 bits 0..2	Version	`Версия UNWIND_INFO, проект поддерживает 1 и 2`
byte0 bits 3..7	Flags	EHANDLER, UHANDLER, CHAININFO
byte1	SizeOfProlog	Размер пролога функции
byte2	CountOfCodes	Сколько unwind slots описывает операции
byte3 bits 0..3	FrameRegister	Регистр frame pointer
byte3 bits 4..7	FrameOffset	Смещение frame pointer, умножается на 16

9.2. Unwind slots

После 4-байтного заголовка идет массив 2-байтных slots. Один логический unwind-op может занимать 1, 2 или 3 slots. Поэтому код сначала определяет, сколько slots занимает операция, и только потом читает operands.

UWOP	Что описывает
`UWOP_PUSH_NONVOL`	Функция сохранила nonvolatile-регистр через push
`UWOP_ALLOC_LARGE`	Функция выделила большой стековый кадр
`UWOP_ALLOC_SMALL`	Функция выделила маленький стековый кадр
`UWOP_SET_FPREG`	Функция использует frame pointer
`UWOP_SAVE_NONVOL`	Функция сохранила nonvolatile-регистр в стек
`UWOP_SAVE_NONVOL_FAR`	То же, но с большим offset
`UWOP_SAVE_XMM128`	Сохранен XMM-регистр
`UWOP_PUSH_MACHFRAME`	Машинный exception frame

9.3. Что складывается в `UnwindSummary`

- valid - удалось ли разобрать `UNWIND_INFO`.

- version, flags, prolog_size, count_codes - поля заголовка.
- frame_reg и frame_off - frame pointer.
- has_handler и handler_rva - есть ли exception handler.
- has_chain и chain - есть ли chained `RUNTIME_FUNCTION`.
- fixed_stack_bytes - примерная фиксированная глубина стека.
- pushed_nonvol_count - сколько nonvolatile-регистров сохранено через push.
- saved_nonvol_mask и saved_xmm_mask - битовые маски сохраненных регистров.
- anomalies - битовая маска проблем.
- parse_error - текст ошибки, если парсинг провалился.

9.4. Handler и chain

В конце `UNWIND_INFO` может быть дополнительная часть. Если стоит EHANDLER или UHANDLER, там лежит `RVA` handler. Если стоит CHAININFO, там лежит еще одна `RUNTIME_FUNCTION`. Проект умеет идти по chain, но ограничивает глубину, чтобы не уйти в бесконечную петлю.

10. Аномалии `UNWIND_INFO`

Аномалия - это не всегда ошибка программы. Это флаг: запись выглядит необычно или нарушает ожидаемую структуру. Часть аномалий влияет на эвристики, часть при strict ABI считается отдельными предупреждениями.

Флаг	Что означает
`UA_VERSION_UNKNOWN`	`Версия UNWIND_INFO не 1 и не 2`
`UA_TRUNCATED`	Данных не хватает, запись обрезана
`UA_BAD_OPCODE`	Неизвестная или неправильная unwind operation
`UA_BAD_OPERAND`	Операнд операции выглядит неправильным
`UA_CODEOFFSET_OUTSIDE`	Code offset выходит за prolog
`UA_CHAIN_LOOP`	Chain зациклился
`UA_CHAIN_DEPTH_EXCEEDED`	Chain глубже разрешенного лимита
`UA_HANDLER_OOB`	`Handler RVA за пределами образа`
`UA_CODEOFFSET_ORDER`	Unwind codes идут в странном порядке
`UA_OPERAND_SANITY`	Размеры/offsets выглядят подозрительно
`UA_ALIGNMENT`	`UNWIND_INFO не выровнен как ожидается`
`UA_CHAIN_INVALID`	`Chained RUNTIME_FUNCTION некорректен`
`UA_ABI_*`	Строгие Microsoft x64 ABI warnings

10.1. Почему ABI warnings отделены

Некоторые production toolchains могут создавать записи, которые формально неидеальны по строгому ABI, но на практике являются совместимыми. Поэтому strict ABI diagnostics видны в отчете, но сами по себе не повышают suspicion score.

11. `heuristics.c`: как проект решает, что выглядит странно

`heuristics.c` не читает файл. Он получает уже готовые структуры: `PeImage`, RuntimeFunction[], `UnwindSummary`[] и us_ok[]. Его задача - посчитать признаки и дать понятный итог: score, suspicion, confidence, verdict, findings.

11.1. Почему эвристики бывают разных типов

Не все можно проверить на одной записи. Например, плохой opcode - локальная проверка одного `UNWIND_INFO`. А reuse одного handler невозможно понять без всей таблицы функций. Поэтому проект смешивает локальные, глобальные, статистические и комбинированные проверки.

Тип	Пример	Что нужно для проверки
`PE`-level	Секция выходит за файл	Весь `PeImage`

Тип	Пример	Что нужно для проверки
Table-level	`RUNTIME_FUNCTION ranges пересекаются`	Вся `.pdata`
Entry-level	`EndAddress <= BeginAddress`	`Одна RUNTIME_FUNCTION`
UNWIND-level	`UNWIND_INFO не парсится`	Один `UnwindSummary`
Placement	`UNWIND_INFO лежит в writable section`	`PeImage` + `UnwindSummary`
Statistical	Один handler доминирует	Все функции сразу
Combo	Writable metadata + writable handler	Несколько счетчиков

11.2. Пять семейств evidence

Family	Что означает простыми словами
`table_integrity`	`Проблемы таблицы .pdata / RUNTIME_FUNCTION`
`unwind_integrity`	`Проблемы структуры или декодирования UNWIND_INFO`
`metadata_placement`	Метаданные или handlers лежат в подозрительных секциях
`unwind_reuse`	`Слишком много функций переиспользуют один UNWIND_INFO`
`handler_dominance`	Один handler обслуживает слишком много функций

11.3. Все 27 finding-кодов

N	Finding	Family	Что означает
1	pe_section_raw_oob	table_integrity	Raw bytes секции выходят за файл
2	pe_section_virtual_oob	table_integrity	Virtual range секции выходит за SizeOfImage
3	pe_section_raw_overlap	table_integrity	Raw ranges секций пересекаются
4	pe_section_virtual_overlap	table_integrity	Virtual ranges секций пересекаются
5	pe_section_misaligned	table_integrity	Нарушены alignment/header invariants
6	pe_exception_directory_oob	table_integrity	Exception Directory повреждена или вне границ
7	no_runtime_functions	table_integrity	Нет .pdata при наличии executable code
8	invalid_ranges	table_integrity	EndAddress <= BeginAddress
9	rf_oob	table_integrity	Begin/End выходят за SizeOfImage
10	unwind_rva_oob	table_integrity	UnwindInfoRVA отсутствует или вне образа
11	rf_unsorted	table_integrity	Таблица не отсортирована по BeginAddress
12	rf_overlaps	table_integrity	Диапазоны функций пересекаются
13	duplicate_begin	table_integrity	Несколько функций с одним BeginAddress
14	unwind_parse_fail	unwind_integrity	UNWIND_INFO не удалось разобрать
15	unwind_anomalies	unwind_integrity	Есть structural anomalies в UNWIND_INFO
16	unwind_reuse_moderate	unwind_reuse	Умеренное reuse подтверждено другим сигналом
17	unwind_reuse_strong	unwind_reuse	Сильное reuse одного UnwindInfoRVA
18	handler_dominance	handler_dominance	Один handler доминирует среди функций
19	unwind_in_exec_section	metadata_placement	UNWIND_INFO лежит в executable section
20	unwind_unusual_section	metadata_placement	UNWIND_INFO лежит не в типичной секции
21	unwind_writable_section	metadata_placement	UNWIND_INFO лежит в writable section
22	handler_nonexec_section	metadata_placement	Handler указывает в non-executable/unknown section
23	handler_writable_section	metadata_placement	Handler указывает в writable section
24	placement_exec_nonexec_handler_combo	metadata_placement	Executable unwind + nonexec handler вместе
25	placement_integrity_combo	unwind_integrity	Странное placement + integrity-проблемы
26	reuse_dominance_combo	unwind_reuse	Reuse UNWIND_INFO + dominant handler
27	writable_metadata_handler_combo	metadata_placement	Writable unwind + writable handler

11.4. Как heuristics_analyze работает внутри

1. Копирует PE-level счетчики из PeImage.
2. Создает hash map для подсчета reuse UnwindInfoRVA.
3. Создает hash map для подсчета reuse handler_rva.
4. Идет по каждой RuntimeFunction.
5. Проверяет Begin/End, выходы за SizeOfImage, UnwindInfoRVA.
6. Считает нарушения сортировки.
7. Увеличивает счетчик reuse для unwind_rva.
8. Если есть handler, увеличивает счетчик reuse для handler_rva.
9. Определяет, в какой секции лежит UNWIND_INFO.
10. Проверяет unusual/executable/writable placement.
11. Если есть handler, проверяет секцию handler.
12. Считает parse_fail, anomalies и ABI warnings.
13. Создает отсортированный массив диапазонов.
14. После сортировки ищет overlaps и duplicate begins.

15 Считает max reuse и ratios.
16 По счетчикам добавляет findings и score.
17 Добавляет combo findings.
18 Вызывает finalize_verdict.

11.5. Score, suspicion, confidence

- score - целое число 0..100, чем выше, тем подозрительнее.
- suspicion - score / 100.
- confidence - уверенность в интерпретации, это не вероятность вредоносности.
- evidence_level - none, weak, medium или strong.
- verdict - текстовый итог: no_strong_static_anomalies, low_suspicion, moderate_suspicion, high_suspicion.

Важно: score не говорит 'это точно вредонос'. Он говорит: 'метаданные выглядят настолько-то подозрительно и требуют такой-то ручной проверки'.

12. `dynamic_win.c`: что делает динамический режим

Статический режим только читает файл. Динамический режим запускает target под Windows Debug API и наблюдает, что происходит при исполнении. Это опаснее, потому что target реально выполняется. Для недоверенных samples нужна isolated VM.

12.1. Зачем нужен dynamic mode

- Понять, реально ли выполняются handlers, найденные статически.
- Увидеть first-chance и second-chance exceptions.
- Увидеть регистрацию VEH, VCH и UEF.
- Увидеть вызовы RtlAddFunctionTable и RtlInstallFunctionTableCallback.
- Собрать короткую RIP trace после handler hit.

12.2. Как работает debug loop

1 Строится command line для CreateProcessA.
2 Target запускается с DEBUG_ONLY_THIS_PROCESS.
3 WaitForDebugEvent ждет события отладчика.
4 На CREATE_PROCESS_DEBUG_EVENT запоминается base address main module.
5 Обновляется список модулей через Toolhelp snapshot.
6 Ставятся API breakpoints на kernel32/ntdll функции.
7 Ставятся breakpoints на handlers из `UNWIND_INFO`.
8 На LOAD_DLL список модулей обновляется и breakpoints пробуются снова.
9 На EXCEPTION_DEBUG_EVENT анализируются breakpoint, single-step и обычные exceptions.
10 На EXIT_PROCESS_DEBUG_EVENT фиксируется exit code.
11 При timeout target завершается.

12.3. Как работает INT3 breakpoint

```
1. ReadProcessMemory читает оригинальный байт по адресу.
2. WriteProcessMemory записывает 0xCC.
3. Target доходит до адреса и ловит EXCEPTION_BREAKPOINT.
4. Проект находит Breakpoint по адресу.
5. Оригинальный байт возвращается назад.
6. RIP откатывается на адрес инструкции.
7. В EFlags ставится Trap Flag.
8. Target делает один шаг.
9. При EXCEPTION_SINGLE_STEP breakpoint ставится снова.
```

12.4. Что пишется в DynamicReport

Поле/группа	Что означает
exceptions_seen	Сколько обычных исключений увидел debug loop
first_chance_seen	Сколько first-chance exceptions
second_chance_seen	Сколько second-chance exceptions
unwind_handler_hits	Попадания в handlers из UNWIND_INFO
veh/vch/uef registrations	Регистрация runtime handlers
veh/vch/uef hits	Попадания в runtime handlers
rtl_add_function_table_calls	Вызовы динамической регистрации function table
events[]	Структурированный журнал runtime events
trace[]	Короткая RIP trace после первого handler hit
instrumentation_complete	Все ли breakpoints/modules/events были собраны без потерь

12.5. Связь static + dynamic

report.c потом смотрит, есть ли корреляция. Например, если статически handler выглядит подозрительно и dynamic mode реально видит handler hit, это сильнее, чем один статический признак. Если metadata placement странное и runtime вызывает RtlAddFunctionTable, это тоже может усиливать case.

13. report.c: как результаты превращаются в отчеты

report.c не парсит PE и не считает эвристики. Он получает готовые структуры и сериализует их в понятные форматы.

Формат	Для чего нужен
Console	Быстро увидеть краткую информацию и findings
CSV	Плоская таблица функций и unwind-сводок
Ghidra CSV	Импорт диапазонов функций и комментариев в Ghidra
JSON	Полный машинный отчет по docs/report.schema.json
HTML	Самодостаточный dashboard для чтения человеком

13.1. JSON

JSON - главный контракт. Он содержит input metadata, PE fields, sections, heuristic_analysis, dynamic_validation, assessment и functions.

JSON верхнего уровня:

```
input
report_metadata
image_base / size_of_image / headers
exception_directory
pe_validation
sections[]
heuristic_analysis
dynamic_validation
assessment
functions[]
```

13.2. Assessment

Assessment - это итоговое соединение static и dynamic. Там есть final_verdict, final_confidence, static_evidence_level, runtime_evidence_level, case_strength, static_dynamic_correlation и manual_validation_steps.

13.3. Почему нужны escape-функции

Любой текст из входного файла, пути или ошибок может содержать кавычки, переносы, невалидный UTF-8, HTML-символы. Поэтому `report.c` имеет json_escape и html_escape. Без этого можно сломать отчет или получить небезопасный HTML.

13.4. Ghidra CSV

Ghidra CSV специально простой: begin_rva, end_rva, name, comment. Скрипт Ghidra читает эти строки, создает функции, добавляет комментарии и bookmarks для ошибок.

14. Владение памятью: кто выделяет и кто освобождает

Для C это критически важно. В проекте почти каждый объект имеет явного владельца.

Объект	Где выделяется	Кто освобождает
bytes input file	`buf_init_from_file`	`main.c` через `buf_free_owned`
`PeImage`.sections	`pe_parse`	`main.c` через `pe_free`
RuntimeFunction* rf	`unwind_read_runtime_functions_ex`	`main.c` через `free`
`UnwindSummary`* us	`calloc` в `main.c`	`main.c` через `free`
`bool* us_ok`	`calloc` в `main.c`	`main.c` через `free`
`HeuristicReport`.details	`heuristics_analyze`	`heuristics_free`
`DynamicReport`.details	`dynamic_report_init`	`dynamic_report_free`
Report `StrBuf` buffers	`report.c` `writer-функции`	та же writer-функция

14.1. Главный принцип

Тот, кто выделил память или получил ownership, должен знать, когда ее освободить. Если функция только смотрит на чужой объект, она не должна его free-ить.

14.2. Почему `report.c` не хранит указатели

Report writers берут модели только на время записи. Они не сохраняют pointers в глобальные переменные. Это уменьшает риск dangling pointer после освобождения.

15. Тесты: как проект доказывает, что не ломается

Тесты здесь очень важны, потому что проект читает недоверенные бинарные данные. Ошибка границ может стать crash или undefined behavior.

Группа	Что проверяет
`tests/unit/unit_unwind.c`	Декодирование `UNWIND_INFO`, операции, chain, ошибки
`tests/unit/unit_core_edges.c`	Пустые таблицы, overflow, mapping, overlaps
`tests/unit/unit_heuristics*.c`	Score, confidence, mitigations, combos
`tests/unit/unit_report_json.c`	JSON serialization и UTF-8 safety
`tests/unit/unit_report_html.c`	HTML escaping и dashboard
tests/samples	Шесть маленьких PE samples с ожидаемыми verdicts
`run_regression_samples.py`	Запускает tool, проверяет JSON Schema и semantic invariants
`report_semantics.py`	Проверяет согласованность полей внутри JSON
`run_mutation_fuzz.py`	Мутационный fuzz плохих входов
`tests/fuzz/fuzz_parser.c`	libFuzzer harness

15.1. Почему нужны regression samples

Unit tests проверяют куски кода. Samples проверяют весь путь: реальный PE -> парсинг -> эвристики -> JSON -> schema -> semantic checks.

16. Как переписывать проект заново и учить C

Не надо переписывать все сразу. Делай маленькие версии, каждая из которых работает и проверяется. Тогда C будет учиться естественно.

16.1. Версия 1: чтение файла и MZ/PE

- Сделай `Buf` и buf_read.
- Открой файл, прочитай bytes.
- Проверь MZ.
- Прочитай e_lfanew.
- Проверь `PE` signature.
- Выведи machine и number of sections.

16.2. Версия 2: секции и RVA mapping

- Прочитай OptionalHeader `PE32+`.
- Прочитай `ImageBase`, `SizeOfImage`, SizeOfHeaders.
- Прочитай таблицу секций.
- Напиши pe_find_section_for_rva.
- Напиши pe_rva_to_fileoff.
- Проверь на нескольких `RVA` руками.

16.3. Версия 3: `.pdata`

- Найди `Exception Directory`.
- Переведи exception_rva в `file offset`.
- Проверь exception_size % 12 == 0.
- Прочитай RuntimeFunction[]
- Напечатай begin/end/unwind для каждой записи.

16.4. Версия 4: UNWIND_INFO header

- Для каждой RuntimeFunction возьми unwind_rva.
- Переведи в `file offset`.
- Прочитай 4 байта заголовка.
- Вытащи version, flags, prolog, count, frame reg/off.
- Печатай краткую строку по каждой функции.

16.5. Версия 5: unwind operations

- Добавь slots_for_code.
- Разбери PUSH_NONVOL, ALLOC_SMALL, ALLOC_LARGE.
- Посчитай fixed_stack_bytes.
- Добавь saved register masks.
- Добавь parse_error и anomalies.

16.6. Версия 6: basic heuristics

- invalid_ranges.
- rf_oob.
- unwind_rva_oob.
- unwind_parse_fail.
- unwind_anomalies.
- простая сумма score.

16.7. Версия 7: отчеты

- Сначала CSV.
- Потом JSON без HTML.
- Потом JSON Schema-compatible поля.
- HTML оставь после JSON.

16.8. Версия 8: продвинутые эвристики

- placement эвристики через pe_find_section_for_rva.
- hash map для reuse.
- handler dominance.
- combo findings.
- confidence и mitigations.

16.9. Версия 9: dynamic mode

- Сначала сделай stub: dynamic mode unsupported.
- Потом CreateProcessA + debug loop без breakpoints.
- Потом module list.
- Потом API breakpoints.
- Потом unwind handler breakpoints.
- Потом trace.

16.10. Минимальная первая цель

```
Первая рабочая программа должна уметь:

ehua_rewrite target.exe --dump

и печатать:
  PE OK
  Sections: N
  Exception Directory RVA/size
  RUNTIME_FUNCTION entries:
    [0] begin end unwind
    [1] begin end unwind
    ...
```

Это уже будет настоящий полезный C-проект. После этого добавлять `UNWIND_INFO` и эвристики будет намного проще.

17. Полный путь выполнения от argv до отчета

Эта глава идет строго по маршруту программы. Читай ее как трассу: кто создает данные, кто их принимает, что проверяет и что передает дальше.

```
argv -> Args -> Buf -> PeImage -> RuntimeFunction[] -> UnwindSummary[] -> HeuristicReport -> optional DynamicReport -> repo
```

17.1. Запуск и argv

Где смотреть: src/`main.c`: main, parse_args.

Программа начинается с обычного argc/argv. `main.c` превращает текстовые параметры в структуру Args: входной файл, режим dump, эвристики, форматы отчетов, dynamic mode, лимиты и strict ABI.

Вход и выход шага

- Вход: argc/argv от ОС.
- Выход: Args с режимами запуска.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. main получает argc/argv
2. parse_args идет слева направо
3. числа проверяются через parse_long_range/parse_u32_range
4. --help и --version завершают работу без анализа
5. ошибки аргументов дают ненулевой exit code

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- Начни переписывание с одного обязательного пути к exe
- Потом добавляй флаги по одному
- Сразу проверяй плохой ввод

17.2. Чтение файла

Где смотреть: src/`util.c`: buf_init_from_file.

Static mode не запускает target. Он читает файл как массив байтов и создает `Buf`: data + size. Отдельный owned_mem остается владельцем выделенной памяти.

Вход и выход шага

- Вход: путь к target`.exe` из Args.
- Выход: `Buf` и owned_mem.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. открыть файл в binary mode
2. узнать размер
3. malloc под весь файл
4. прочитать байты
5. заполнить `Buf` и вернуть owned_mem

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- `Buf` - только вид на байты
- owned_mem надо освободить
- это первый урок владения памятью в C

17.3. Безопасное чтение чисел

Где смотреть: src/`util.c`/include/util.h: buf_read_u16/u32/u64.

Проект не приводит байты к struct через cast. Он читает little-endian числа маленькими функциями, которые сначала проверяют границы.

Вход и выход шага

- Вход: `Buf` с байтами файла.
- Выход: безопасно прочитанные little-endian числа.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. buf_read проверяет off + len <= size
2. buf_read_u32 копирует 4 байта
3. le32 собирает число из байтов
4. false поднимается наверх как ошибка
5. парсер не читает за концом файла

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- Не начинай с IMAGE_DOS_HEADER cast
- Сделай read_u8/u16/u32
- Проверь на маленьком файле, что границы работают

17.4. DOS header MZ

Где смотреть: src/pe.c: read_dos_lfanew, pe_parse.

Первый форматный признак PE - MZ в начале файла. Поле e_lfanew говорит, где находится настоящий PE-заголовок.

Вход и выход шага

- Вход: первые байты Buf.
- Выход: e_lfanew или ошибка формата.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. проверить байты MZ
2. прочитать e_lfanew по offset 0x3c
3. проверить, что e_lfanew внутри файла
4. перейти к PE signature
5. при ошибке остановить pe_parse

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- Нарисуй файл на бумаге: offset 0 -> MZ, offset 0x3c -> e_lfanew
- Это хорошая первая самостоятельная функция парсера

17.5. PE signature и COFF

Где смотреть: src/pe.c: pe_parse.

По e_lfanew проект проверяет PE\0\0 и COFF-заголовок. Для этого анализатора нужна архитектура AMD64, потому что x64 unwind устроен через .pdata/UNWIND_INFO.

Вход и выход шага

- Вход: offset PE header.
- Выход: COFF-сведения и число секций.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. прочитать PE signature
2. проверить Machine AMD64
3. прочитать NumberOfSections
4. прочитать SizeOfOptionalHeader
5. проверить, что таблица секций помещается в файл

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- Поддерживай сначала только PE32+ AMD64
- Другие архитектуры добавлять позже
- Так проще не смешать форматы

17.6. Optional Header PE32+

Где смотреть: src/pe.c: pe_parse.

Optional Header хранит ImageBase, SizeOfImage, alignments и DataDirectory. Самое важное поле для проекта - Exception Directory RVA/size.

Вход и выход шага

- Вход: Optional Header bytes.
- Выход: ImageBase, SizeOfImage, Exception Directory.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. проверить magic PE32+
2. прочитать ImageBase
3. прочитать SectionAlignment/FileAlignment
4. прочитать SizeOfImage
5. взять Exception Directory из DataDirectory

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- Здесь появляется разница между RVA и file offset
- Все дальнейшие адреса надо переводить через секции

17.7. Таблица секций

Где смотреть: src/`pe.c`: `PeSection`.

Секции - карта файла. По ним проект понимает, куда попадает `RVA`, где данные лежат на диске и какие права у области: executable, writable, readable.

Вход и выход шага

- Вход: section table bytes.
- Выход: `PeSection`[] и счетчики проблем секций.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. выделить `PeSection`[]
2. скопировать имя секции
3. сохранить VirtualAddress/VirtualSize
4. сохранить RawPtr/RawSize
5. посчитать raw/virtual/oob/overlap/alignment problems

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- Это отличный учебный блок: массив структур, malloc/free, строки фиксированной длины, битовые flags

17.8. RVA -> file offset

Где смотреть: src/pe.c: pe_find_section_for_rva, pe_rva_to_fileoff.

Центральный перевод проекта: RVA из PE metadata превращается в позицию байта в файле. Без этого нельзя прочитать ни .pdata, ни UNWIND_INFO.

Вход и выход шага

- Вход: PeImage + RVA.
- Выход: file offset для чтения байтов.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. найти секцию, где RVA попадает в virtual range
2. посчитать delta = RVA - VirtualAddress
3. offset = RawPtr + delta
4. проверить offset внутри файла
5. вернуть false, если RVA не отображается

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- Пока это не понял, проект кажется магией
- Сделай ручной пример с VA=0x3000 RawPtr=0x1200 RVA=0x3120

17.9. `Exception Directory`

Где смотреть: src/`pe.c` + src/`unwind.c`.

Проект не ищет секцию по имени `.pdata` как главный источник. Он берет `Exception Directory` из Optional Header. Обычно она указывает на `.pdata`.

Вход и выход шага

- Вход: `PeImage`.exception_rva/size.
- Выход: диапазон `.pdata`/`RUNTIME_FUNCTION` table.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. pe_parse сохраняет exception_rva/exception_size
2. `unwind.c` переводит exception_rva в offset
3. проверяет size >= 12
4. проверяет size % 12
5. считает количество `RUNTIME_FUNCTION`

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- Твоя формулировка почти верна: путь начинается с `Exception Directory`, а не просто с имени `.pdata`

17.10. RUNTIME_FUNCTION[]

Где смотреть: src/unwind.c: unwind_read_runtime_functions_ex.

Одна запись RUNTIME_FUNCTION занимает 12 байт: BeginAddress, EndAddress, UnwindInfoRVA. Это оглавление функций и ссылок на UNWIND_INFO.

Вход и выход шага

- Вход: байты Exception Directory.
- Выход: RuntimeFunction[].
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. выделить RuntimeFunction[]
2. прочитать begin_rva
3. прочитать end_rva
4. прочитать unwind_rva
5. сохранить count

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- .pdata не хранит сами unwind-команды
- Она хранит диапазоны кода и указатель на описание восстановления стека

17.11. Диапазоны функций

Где смотреть: src/`heuristics.c`: invalid_ranges, overlaps.

`BeginAddress`/`EndAddress` должны описывать нормальный диапазон кода. Если begin >= end, диапазон выходит за образ или пересекается с другим, это становится finding.

Вход и выход шага

- Вход: RuntimeFunction[].
- Выход: счетчики проблем диапазонов.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. парсер сохраняет записи как есть
2. heuristics считает invalid ranges
3. проверяет выход за `SizeOfImage`
4. сортирует индексы по begin
5. ищет overlaps и duplicate begin

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- Парсер извлекает данные
- Эвристика решает, подозрительно это или нет
- Не смешивай эти роли

17.12. Переход к UNWIND_INFO

Где смотреть: src/unwind.c: unwind_parse.

Для каждой RUNTIME_FUNCTION берется UnwindInfoRVA. Обычно он ведет в .xdata, но код проверяет фактическую секцию, а не верит названию.

Вход и выход шага

- Вход: RuntimeFunction.unwind_rva.
- Выход: offset UNWIND_INFO и секция metadata.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. взять rf[i].unwind_rva
2. перевести RVA в file offset
3. прочитать первые 4 байта UNWIND_INFO
4. если RVA не отображается, выставить ошибку
5. запомнить секцию для placement-эвристик

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- Правильная фраза: .pdata указывает на UNWIND_INFO, который обычно лежит в .xdata

17.13. Заголовок UNWIND_INFO

Где смотреть: src/unwind.c: parse_unwindinfo_header.

UNWIND_INFO начинается с компактного заголовка: version, flags, prolog size, count of codes, frame register и frame offset.

Вход и выход шага

- Вход: первые 4 байта UNWIND_INFO.
- Выход: version, flags, prolog, count codes, frame fields.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. version = byte & 7
2. flags = byte >> 3
3. прочитать prolog_size
4. прочитать count_codes
5. разобрать frame_reg/frame_off

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- Это место учит битовым операциям
- Один байт хранит несколько логических полей

17.14. Unwind slots

Где смотреть: src/`unwind.c`: slots_for_code, unwind_parse.

После заголовка идет массив 2-байтовых slots. Это маленький байткод пролога функции: push register, stack allocation, save register, push machine frame.

Вход и выход шага

- Вход: `UNWIND_INFO` slots.
- Выход: последовательность unwind operations.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. прочитать slot
2. вытащить CodeOffset/UnwindOp/OpInfo
3. понять длину операции
4. прочитать дополнительные slots
5. перейти к следующей операции

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- Сначала реализуй PUSH_NONVOL и ALLOC_SMALL
- Потом ALLOC_LARGE и SAVE_*
- Так проще не утонуть

17.15. Стек и регистры

Где смотреть: src/`unwind.c`: fixed_stack_bytes, masks.

Проект превращает unwind-команды в смысл: сколько фиксированного стека выделено и какие nonvolatile/XMM-регистры сохранены.

Вход и выход шага

- Вход: unwind operations.
- Выход: fixed_stack_bytes и register masks.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. PUSH_NONVOL добавляет 8 байт
2. ALLOC_SMALL/ALLOC_LARGE увеличивают stack allocation
3. SAVE_NONVOL отмечает бит в mask
4. SAVE_XMM128 отмечает XMM mask
5. summary сохраняется в `UnwindSummary`

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- Маски регистров - классический C-прием
- Один uint16 может хранить набор флагов

17.16. Handler и chain

Где смотреть: src/`unwind.c`: handler_rva, parse_chain_rf.

После slots может идти handler `RVA` или chained `RUNTIME_FUNCTION`. Это зависит от flags. Chain можно читать дальше, но нужен лимит глубины.

Вход и выход шага

- Вход: flags + tail после slots.
- Выход: handler_rva или chained RuntimeFunction.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. выровнять количество slots до четного
2. если EHANDLER/UHANDLER - прочитать handler_rva
3. если CHAININFO - прочитать chained RF
4. проверить handler внутри образа
5. остановить chain loop/depth

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- Chain можно добавить позже
- Сначала сделай no-follow-chain режим
- Потом добавь depth limit

17.17. Anomaly flags

Где смотреть: include/unwind.h + src/`unwind.c`.

Если `UNWIND_INFO` частично странный, проект не обязан выбрасывать всю запись. Он ставит биты anomaly: bad opcode, truncated, handler_oob, chain_loop, ABI warning.

Вход и выход шага

- Вход: результат парсинга `UNWIND_INFO`.
- Выход: parse status и anomaly bitmask.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. критический сбой дает us_ok=false
2. мягкая странность дает anomalies != 0
3. `report.c` переводит биты в текст
4. heuristics считает долю anomalies
5. strict ABI добавляет более строгие проверки

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- Это хороший паттерн C: сохранить частичный результат и отдельно статус/flags

17.18. Массивы rf/us/us_ok

Где смотреть: src/`main.c`.

main держит три связанных массива: rf[i] - запись `.pdata`, us[i] - разобранный `UNWIND_INFO`, us_ok[i] - статус разбора.

Вход и выход шага

- Вход: rf_count.
- Выход: связанные массивы rf/us/us_ok.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. выделить rf
2. выделить us
3. выделить us_ok
4. цикл по i вызывает unwind_parse
5. тот же индекс идет в reports и heuristics

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- Один индекс связывает разные слои данных
- Так проще передавать модель в отчеты

17.19. --dump

Где смотреть: src/`main.c` + src/`report.c`.

Dump - учебный режим. Он показывает, что реально прочитано: begin/end/unwind, flags, стек, регистры, handler, anomalies.

Вход и выход шага

- Вход: rf/us/us_ok.
- Выход: читаемый console dump.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. напечатать brief `PE`
2. идти по записям до limit
3. format flags
4. format registers
5. показать parse status

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- Сделай --dump раньше эвристик
- Пока не видишь сырые данные, score выглядит непонятно

17.20. Старт эвристик

Где смотреть: src/`heuristics.c`: heuristics_analyze.

Эвристики получают уже готовую модель: `PeImage`, RuntimeFunction[], `UnwindSummary`[], us_ok[]. Они почти не читают байты файла.

Вход и выход шага

- Вход: `PeImage` + rf/us/us_ok.
- Выход: `HeuristicReport`.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. инициализировать `HeuristicReport`
2. перенести `PE` section issues
3. пройти по rf/us
4. посчитать локальные counters
5. запустить глобальные проверки

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- Такой файл легко тестировать на искусственных данных
- Поэтому есть отдельные unit_heuristics tests

17.21. Integrity эвристики

Где смотреть: src/`heuristics.c`: table/unwind integrity.

Эта семья проверяет, согласована ли сама таблица `.pdata` и корректны ли `UNWIND_INFO` записи.

Вход и выход шага

- Вход: `HeuristicReport` counters.
- Выход: integrity findings.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. invalid_ranges
2. rf_oob
3. unwind_rva_oob
4. rf_unsorted/overlaps
5. unwind_parse_fail/anomalies

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- Начни эвристики именно отсюда
- Они проще placement и dynamic checks

17.22. Placement эвристики

Где смотреть: src/`heuristics.c`: section placement.

`UNWIND_INFO` обычно является данными и лежит в `.xdata`/`.rdata`/`.pdata`. Handler обычно является кодом. Проект проверяет, не перепутаны ли эти роли.

Вход и выход шага

- Вход: `RVA` metadata + section flags.
- Выход: placement findings.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. найти секцию unwind_rva
2. проверить имя секции
3. проверить executable
4. проверить writable
5. найти секцию handler_rva

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- `UNWIND_INFO` - данные
- handler - код
- writable metadata особенно подозрительна

17.23. Reuse и dominance

Где смотреть: src/`heuristics.c`: HMap.

Проект считает, не слишком ли много функций используют один `UNWIND_INFO` или один handler. Это статистический сигнал искусственной схемы.

Вход и выход шага

- Вход: unwind_rva/handler_rva frequencies.
- Выход: reuse и dominance findings.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. hmap_inc для unwind_rva
2. hmap_inc для handler_rva
3. найти max entry
4. посчитать долю
5. добавить moderate/strong finding

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- Один повтор не страшен
- Массовое повторение может быть сигналом упаковщика или подмены

17.24. Score, confidence, verdict

Где смотреть: src/`heuristics.c`: finalize_verdict.

Findings превращаются в итог: score 0..100, evidence families, confidence и verdict. Это не вероятность malware, а инженерная оценка подозрительности EH/unwind metadata.

Вход и выход шага

- Вход: findings + families.
- Выход: score, confidence, verdict.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. каждая finding добавляет score
2. family mask показывает разнообразие сигналов
3. confidence растет от strong/medium/families
4. mitigations снижают уверенность
5. thresholds дают low/moderate/high verdict

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- Не делай вывод по одной finding
- Смотри score вместе с confidence и details

17.25. Dynamic mode

Где смотреть: src/`dynamic_win.c`.

Dynamic mode на Windows запускает target под отладчиком и собирает runtime evidence: VEH/VCH/UEF, RtlAddFunctionTable, handler hits, exceptions.

Вход и выход шага

- Вход: target process under Debug API.
- Выход: `DynamicReport`.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. CreateProcessA с DEBUG_ONLY_THIS_PROCESS
2. WaitForDebugEvent loop
3. список модулей через Toolhelp
4. software breakpoints 0xCC
5. `DynamicReport` events/trace/details

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- Это изучай последним
- Сначала static pipeline должен быть полностью понятен

17.26. Отчеты и завершение

Где смотреть: src/`report.c` + src/`main.c`.

В конце `report.c` сериализует готовую модель, а main освобождает память. CSV/Ghidra CSV удобны для таблиц, JSON для машинной проверки, HTML для человека.

Вход и выход шага

- Вход: готовая модель анализа.
- Выход: CSV, Ghidra CSV, JSON, HTML и cleanup.
- Если шаг не может надежно продолжать работу, он возвращает ошибку или помечает запись как подозрительную, а следующий слой получает уже явный статус.

Что происходит по шагам

1. report_model_valid проверяет согласованность
2. CSV пишет функции
3. JSON пишет schema/metadata/findings/functions
4. HTML пишет самодостаточный отчет
5. cleanup вызывает pe_free/heuristics_free/dynamic_report_free/free/buf_free_owned

Почему это отдельный слой

Так проект не смешивает чтение байтов, построение модели, оценку подозрительности и форматирование отчета. Это делает код длиннее, зато его можно проверять по частям.

Что важно для переписывания

- `report.c` - слой форматирования
- HTML оставь на конец переписывания
- Всегда проверяй, что каждый malloc имеет free

18. Подробная карта файлов и функций

Счетчик строк ниже берется из реальных файлов проекта при сборке PDF. Это помогает видеть вес модулей и понимать, куда смотреть первым.

Файл	Строк	Ответственность
src/report.c	1363	JSON/HTML/CSV, assessment, manual validation steps.
src/dynamic_win.c	1253	Windows Debug API, breakpoints, DynamicReport.
src/heuristics.c	1044	27 findings, score, confidence, verdict.
src/unwind.c	681	.pdata, RUNTIME_FUNCTION, UNWIND_INFO.
src/main.c	527	CLI, orchestration, ownership, exit codes.
src/util.c	348	Buf, StrBuf, atomic write, SHA-256.
src/pe.c	301	PE32+ headers, sections, RVA mapping.
src/app_info.c	68	Version, schema, compiler, platform, timestamp.
include/unwind.h	114	RuntimeFunction, UnwindSummary, anomaly flags.
include/heuristics.h	109	HeuristicReport and findings model.
include/dynamic.h	122	DynamicReport model.
include/report.h	72	Report writer API.
include/pe.h	67	PeImage and PeSection API.
include/util.h	72	Buf, StrBuf and binary read API.
include/app_info.h	24	Build/report metadata API.

18.1. src/main.c: Диспетчер

Размер основного файла: 527 строк.

Ключевые функции

- parse_args
- build_invocation
- buf_init_from_file
- pe_parse
- unwind_read_runtime_functions_ex
- unwind_parse
- heuristics_analyze
- dynamic_collect_windows
- report_write_*

Главные данные

- Args
- RuntimeFunction* rf
- `UnwindSummary`* us
- bool* us_ok
- `HeuristicReport`
- `DynamicReport`

Связь с архитектурой

- `main.c` связывает модули, но не должен дублировать их внутренний парсинг
- здесь видно владение памятью и порядок cleanup

18.2. src/util.c / include/util.h: Низкий уровень

Размер основного файла: 348 строк.

Ключевые функции

- buf_init_from_file
- buf_read
- buf_read_u8/u16/u32/u64
- sb_init/sb_appendf/sb_free
- file_write_atomic
- sha256_hex

Главные данные

- `Buf`
- `StrBuf`
- owned_mem
- failed flag

Связь с архитектурой

- это лучший старт для изучения C в проекте
- malloc/realloc/free, FILE*, va_list и little-endian чтение

18.3. src/pe.c / include/pe.h: PE parser

Размер основного файла: 301 строк.

Ключевые функции

- pe_parse
- pe_free
- pe_find_section_for_rva
- pe_rva_to_fileoff

Главные данные

- `PeImage`
- `PeSection`
- exception_rva
- exception_size
- section issue counters

Связь с архитектурой

- `pe.c` строит карту файла
- все переходы по `RVA` должны идти через `PeImage`

18.4. src/unwind.c / include/unwind.h: Unwind parser

Размер основного файла: 681 строк.

Ключевые функции

- unwind_read_runtime_functions_ex
- unwind_parse
- parse_unwindinfo_header
- slots_for_code
- unwind_reg_name
- unwind_flags_to_string

Главные данные

- RuntimeFunction
- `UnwindSummary`
- UnwindAnomaly
- chain
- handler_rva

Связь с архитектурой

- ядро предметной области
- похоже на декодер маленького байткода пролога функции

18.5. src/`heuristics.c` / include/heuristics.h: Эвристики

Размер основного файла: 1044 строк.

Ключевые функции

- heuristics_analyze
- add_finding
- annotate_finding
- hmap_inc
- hmap_max_entry
- finalize_verdict
- heuristics_free

Главные данные

- `HeuristicReport`
- HeuristicFinding
- FindingLocation
- evidence families
- mitigations

Связь с архитектурой

- не читает файл напрямую
- работает с моделью и поэтому хорошо тестируется unit-тестами

18.6. src/report.c / include/report.h: Отчеты

Размер основного файла: 1363 строк.

Ключевые функции

- report_print_brief
- report_print_heuristics
- report_write_csv
- report_write_ghidra_csv
- report_write_json_ex
- report_write_html_ex

Главные данные

- ReportMetadata
- AssessmentSummary
- ManualValidationSteps

Связь с архитектурой

- много строк из-за сериализации
- начинай переписывание вывода с CSV, потом JSON, HTML последним

18.7. src/dynamic_win.c / include/dynamic.h: Runtime validation

Размер основного файла: 1253 строк.

Ключевые функции

- dynamic_collect_windows
- mods_refresh_toolhelp
- local_proc_loc
- bp_add_and_arm
- dynamic_event_add
- dynamic_report_free

Главные данные

- `DynamicReport`
- Breakpoint
- PendingRearmList
- ModList
- DynamicEvent

Связь с архитектурой

- слой WinAPI и Debug API
- изучать после static pipeline

18.8. src/app_info.c / include/app_info.h: Metadata сборки

Размер основного файла: 68 строк.

Ключевые функции

- app_version
- app_report_schema_version
- app_build_compiler
- app_build_platform
- app_build_mode
- app_utc_timestamp

Главные данные

- version strings
- schema version
- timestamp

Связь с архитектурой

- маленький файл для изучения заголовков, реализаций и условной компиляции

19. Все 27 эвристик простым языком

Эвристика - это не приговор. Это инженерный сигнал. Его надо читать вместе с score, severity, confidence, details, количеством функций и dynamic evidence. Ниже каждая finding разобрана отдельно.

19.1. pe_section_raw_oob

Что проверяет: raw-диапазон секции выходит за пределы файла.

Откуда данные: `PeImage` counters из `pe.c`.

Почему это подозрительно: байты, на которые указывает секция, физически отсутствуют.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.2. pe_section_virtual_oob

Что проверяет: виртуальный диапазон секции выходит за `SizeOfImage`.

Откуда данные: `PeImage` counters из `pe.c`.

Почему это подозрительно: `RVA`-карта образа становится недостоверной.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.3. pe_section_raw_overlap

Что проверяет: секции пересекаются на диске.

Откуда данные: raw ranges секций.

Почему это подозрительно: одни байты начинают принадлежать двум секциям.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.4. pe_section_virtual_overlap

Что проверяет: секции пересекаются в виртуальной карте.

Откуда данные: virtual ranges секций.

Почему это подозрительно: один RVA может трактоваться неоднозначно.

Как проходит через проект

1. pe.c или unwind.c строит модель
2. heuristics.c считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. report.c выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным RVA/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.5. pe_section_misaligned

Что проверяет: alignment секции выглядит неправильным.

Откуда данные: SectionAlignment/FileAlignment.

Почему это подозрительно: нарушение выравнивания часто идет от упаковщиков или ручной правки.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.6. pe_exception_directory_oob

Что проверяет: `Exception Directory` выходит за границы.

Откуда данные: exception_rva/exception_size.

Почему это подозрительно: `.pdata` нельзя надежно прочитать.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.7. no_runtime_functions

Что проверяет: нет `RUNTIME_FUNCTION` записей.

Откуда данные: rf_count == 0.

Почему это подозрительно: для x64 `PE` с кодом это может быть странно.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.8. invalid_ranges

Что проверяет: `BeginAddress` >= `EndAddress`.

Откуда данные: проход по rf[i].

Почему это подозрительно: функция не может иметь обратный или нулевой диапазон.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.9. rf_oob

Что проверяет: диапазон функции выходит за `SizeOfImage`.

Откуда данные: begin/end против `PeImage`.

Почему это подозрительно: `.pdata` ссылается на код вне образа.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.10. unwind_rva_oob

Что проверяет: `UnwindInfoRVA` нельзя отобразить.

Откуда данные: pe_rva_to_fileoff.

Почему это подозрительно: `.pdata` обещает `UNWIND_INFO`, но байтов нет.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.11. rf_unsorted

Что проверяет: `RUNTIME_FUNCTION` не отсортированы.

Откуда данные: соседние begin_rva.

Почему это подозрительно: обычная `.pdata` ожидается упорядоченной.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.12. rf_overlaps

Что проверяет: диапазоны функций пересекаются.

Откуда данные: сортировка по begin_rva.

Почему это подозрительно: несколько функций претендуют на один код.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.13. duplicate_begin

Что проверяет: несколько записей имеют один `BeginAddress`.

Откуда данные: сравнение после сортировки.

Почему это подозрительно: начало функции становится неоднозначным.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.14. unwind_parse_fail

Что проверяет: `UNWIND_INFO` не разобрался.

Откуда данные: us_ok[i] == false.

Почему это подозрительно: metadata есть, но формат сломан или обрезан.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.15. unwind_anomalies

Что проверяет: `UNWIND_INFO` содержит anomaly flags.

Откуда данные: `UnwindSummary`.anomalies.

Почему это подозрительно: формат частично понятен, но внутри есть странности.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.16. unwind_reuse_moderate

Что проверяет: один `UNWIND_INFO` используется заметно часто.

Откуда данные: hash map по unwind_rva.

Почему это подозрительно: повтор может быть нормальным, но высокая доля требует внимания.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.17. unwind_reuse_strong

Что проверяет: один `UNWIND_INFO` используется массово.

Откуда данные: threshold high по reuse.

Почему это подозрительно: таблица может быть синтетической.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.18. handler_dominance

Что проверяет: один handler доминирует.

Откуда данные: hash map по handler_rva.

Почему это подозрительно: слишком много функций ведут к одному обработчику.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.19. unwind_in_exec_section

Что проверяет: `UNWIND_INFO` лежит в executable секции.

Откуда данные: flags секции unwind_rva.

Почему это подозрительно: unwind metadata - данные, а не код.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.20. unwind_unusual_section

Что проверяет: `UNWIND_INFO` лежит в необычной секции.

Откуда данные: имя секции против allowlist.

Почему это подозрительно: нестандартное размещение может быть маскировкой или упаковщиком.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.21. unwind_writable_section

Что проверяет: `UNWIND_INFO` лежит в writable секции.

Откуда данные: IMAGE_SCN_MEM_WRITE.

Почему это подозрительно: metadata можно менять во время выполнения.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.22. handler_nonexec_section

Что проверяет: handler лежит в non-executable секции.

Откуда данные: секция handler_rva.

Почему это подозрительно: handler должен быть исполняемым кодом.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.23. handler_writable_section

Что проверяет: handler лежит в writable секции.

Откуда данные: секция handler_rva.

Почему это подозрительно: writable handler выглядит опаснее обычного кода.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.24. placement_exec_nonexec_handler_combo

Что проверяет: сочетание проблем metadata placement и handler placement.

Откуда данные: комбинация findings.

Почему это подозрительно: разные placement-сигналы усиливают друг друга.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.25. placement_integrity_combo

Что проверяет: placement-проблемы вместе с поломкой структуры.

Откуда данные: families metadata_placement + integrity.

Почему это подозрительно: одновременно странное место и сломанная таблица дают более устойчивый вывод.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.26. reuse_dominance_combo

Что проверяет: reuse `UNWIND_INFO` вместе с handler dominance.

Откуда данные: reuse + dominance counters.

Почему это подозрительно: массовый шаблон metadata и один handler похожи на искусственную схему.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

19.27. writable_metadata_handler_combo

Что проверяет: writable metadata вместе с writable handler.

Откуда данные: writable checks.

Почему это подозрительно: runtime-подмена становится более правдоподобной.

Как проходит через проект

1. `pe.c` или `unwind.c` строит модель
2. `heuristics.c` считает счетчик или место проблемы
3. add_finding создает code/severity/family/score
4. `report.c` выводит finding в console/JSON/HTML
5. finalize_verdict учитывает всю совокупность сигналов

Как проверить при переписывании

- сделай минимальный счетчик
- добавь synthetic sample или unit test
- проверь details с конкретным `RVA`/section
- убедись, что один слабый сигнал не дает слишком уверенный verdict

20. Как учить C через переписывание этого проекта

Не переписывай все сразу. Делай маленькие версии. Каждая версия должна компилироваться, запускаться и показывать один новый полезный результат.

20.1. Минимальный CLI

Что реализовать

- int main
- argc/argv
- help/version
- exit codes

Критерий готовности

ты понимаешь вход программы.

Какие темы C тренируются

- структуры
- указатели
- массивы и границы
- malloc/free
- битовые операции
- разделение .h и .c
- ошибки через return code

20.2. Buf и чтение файла

Что реализовать

- binary fopen
- file size
- malloc
- read_u32
- bounds checks

Критерий готовности

ты безопасно работаешь с байтами.

Какие темы C тренируются

- структуры
- указатели
- массивы и границы
- malloc/free
- битовые операции
- разделение .h и .c
- ошибки через return code

20.3. Минимальный PE

Что реализовать

- MZ
- e_lfanew
- PE signature
- Machine AMD64
- NumberOfSections

Критерий готовности

ты отличаешь подходящий PE.

Какие темы C тренируются

- структуры
- указатели
- массивы и границы
- malloc/free
- битовые операции
- разделение .h и .c
- ошибки через return code

20.4. Секции и RVA

Что реализовать

- `PeSection`[]
- VirtualAddress
- RawPtr
- Characteristics
- pe_rva_to_fileoff

Критерий готовности

ты умеешь находить данные по `RVA`.

Какие темы C тренируются

- структуры
- указатели
- массивы и границы
- malloc/free
- битовые операции
- разделение .h и .c
- ошибки через return code

20.5. `.pdata`

Что реализовать

- `Exception Directory`
- size % 12
- RuntimeFunction[]
- begin/end/unwind

Критерий готовности

ты видишь список функций.

Какие темы C тренируются

- структуры
- указатели
- массивы и границы
- malloc/free
- битовые операции
- разделение .h и .c
- ошибки через return code

20.6. Минимальный UNWIND_INFO

Что реализовать

- version
- flags
- prolog
- count codes
- PUSH_NONVOL
- ALLOC_SMALL

Критерий готовности

ты видишь смысл stack metadata.

Какие темы C тренируются

- структуры
- указатели
- массивы и границы
- malloc/free
- битовые операции
- разделение .h и .c
- ошибки через return code

20.7. Полный UNWIND_INFO

Что реализовать

- ALLOC_LARGE
- SAVE_NONVOL
- SAVE_XMM128
- PUSH_MACHFRAME
- handler
- chain

Критерий готовности

твой parser становится похож на оригинал.

Какие темы C тренируются

- структуры
- указатели
- массивы и границы
- malloc/free
- битовые операции
- разделение .h и .c
- ошибки через return code

20.8. Первые эвристики

Что реализовать

- invalid_ranges
- rf_oob
- unwind_rva_oob
- parse_fail

Критерий готовности

программа начинает делать выводы.

Какие темы C тренируются

- структуры
- указатели
- массивы и границы
- malloc/free
- битовые операции
- разделение .h и .c
- ошибки через return code

20.9. Отчеты

Что реализовать

- HeuristicFinding
- CSV
- JSON
- HTML
- schema version

Критерий готовности

результат можно сохранять и сравнивать.

Какие темы C тренируются

- структуры
- указатели
- массивы и границы
- malloc/free
- битовые операции
- разделение .h и .c
- ошибки через return code

20.10. Dynamic mode

Что реализовать

- CreateProcessA
- WaitForDebugEvent
- modules
- breakpoints
- `DynamicReport`

Критерий готовности

ты добавляешь runtime evidence.

Какие темы C тренируются

- структуры
- указатели
- массивы и границы
- malloc/free
- битовые операции
- разделение .h и .c
- ошибки через return code

21. Глоссарий простыми словами

Термин	Объяснение
PE	Формат исполняемых файлов Windows
PE32+	64-битная разновидность PE
RVA	Адрес относительно начала загруженного образа
file offset	Позиция байта в файле на диске
section	Именованный диапазон данных в PE
.pdata	Таблица RUNTIME_FUNCTION для x64 exception/unwind
.xdata	Обычно хранит UNWIND_INFO
RUNTIME_FUNCTION	Запись begin/end/unwind_rva
UNWIND_INFO	Описание восстановления стека и регистров
handler	Код, который участвует в обработке исключения
chain	UNWIND_INFO ссылается на другую RUNTIME_FUNCTION
heuristic	Инженерное правило подозрительности
finding	Конкретный найденный признак
score	Суммарная оценка подозрительности 0..100
confidence	Уверенность интерпретации, не вероятность malware
dynamic validation	Запуск target под отладчиком и сбор runtime evidence

22. Что запомнить перед чтением кода

- Проект не исполняет target в static mode.
- Главная связка: .pdata -> RUNTIME_FUNCTION -> UnwindInfoRVA -> UNWIND_INFO.
- UNWIND_INFO обычно лежит в .xdata, но проект проверяет реальную секцию.
- Эвристики бывают локальные, глобальные, статистические и комбинированные.
- main.c держит порядок и освобождение памяти.
- util.c и pe.c - лучшие первые файлы для изучения C.
- dynamic_win.c оставь на конец, это самый сложный системный слой.